

Module 5

Data Files: Opening And Closing A Data File, Reading And Writing A Data File, Processing A Data File, File Handling Functions, Formatted Input And Output Functions, File Processing Programs, Concept Of Binary Files. Additional Features Of C: Enumeration, Bitwise Operators, Command Line Parameters, Macros(Definition, Function Macros, Conditional Macros).

Data Files in C

A data file is a collection of related information stored permanently on a secondary storage device. Unlike variables, the data stored in a file is not lost when the program terminates. C provides several functions for creating, opening, reading, writing, and closing files. File handling is useful when a program needs to store a large amount of information permanently.

- A data file is used to store information permanently so that it can be accessed even after the program has stopped executing.
- Files are generally stored on secondary storage devices such as hard disks, SSDs, or other storage media.
- C provides a special data type called FILE for handling files in a program.
- A file pointer of type FILE * is used to establish communication between the program and the file.
- Data files are useful in applications such as student management systems, payroll systems, banking systems, and inventory systems.
- Files can contain different types of information such as characters, numbers, strings, and records.
- C supports both text files and binary files for storing information.
- Text files store information in a form that can generally be viewed and understood by humans.
- Binary files store information in a machine-oriented binary representation and are useful for efficient storage of structured data.
- File handling allows a program to store, retrieve, search, modify, and process data whenever required.

Opening and Closing a Data File

Before performing any operation on a file, the file must be opened using the `fopen()` function. After completing the required operations, the file should be closed using the `fclose()` function.

Opening a File

The general syntax is:

```
FILE *fp;
```

```
fp = fopen("filename", "mode");
```

For example:

```
FILE *fp;
```

```
fp = fopen("student.txt", "r");
```

Here, fp is the file pointer, student.txt is the file name, and r specifies that the file is opened for reading.

Example

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *fp;
```

```
    fp = fopen("student.txt", "w");
```

```
    if (fp == NULL)
```

```
    {
```

```
        printf("File cannot be opened.");
```

```
        return 1;
```

```
    }
```

```
    printf("File opened successfully.");
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```

File Opening Modes

Mode Description

r Opens an existing file for reading.

w Opens a file for writing and creates it if it does not exist.

a Opens a file for adding new data at the end.

r+ Opens an existing file for both reading and writing.

w+ Opens a file for reading and writing and removes previous contents.

a+ Opens a file for reading and appending.

rb Opens a binary file for reading.

wb Opens a binary file for writing.

Mode Description

ab Opens a binary file for appending.

Important Points

- A file must normally be opened before the program can read data from it or write data into it.
- The `fopen()` function is used to open a file and returns a file pointer.
- The file pointer provides a connection between the C program and the file stored on the system.
- The second argument of `fopen()` specifies the operation that will be performed on the file.
- The `r` mode is used when the program only needs to read an existing file.
- The `w` mode is used to write data into a file and can remove the previous contents of an existing file.
- The `a` mode allows new data to be added at the end of an existing file without removing its previous contents.
- If a file cannot be opened, `fopen()` returns `NULL`, and the program should check this condition.
- The `fclose()` function is used to close an opened file after completing the required operations.
- Closing a file is important because it releases system resources and ensures that pending data is properly written to the file.

Reading and Writing a Data File

Reading means retrieving information from a file into a program, while writing means transferring information from the program into a file. C provides different functions for reading and writing characters, strings, and formatted data.

Writing a Character Using `fputc()`

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *fp;
```

```
    fp = fopen("data.txt", "w");
```

```
    if (fp == NULL)
```

```
        return 1;
```

```
    fputc('A', fp);
```

```
    fputc('B', fp);
```

```

    fputc('C', fp);

    fclose(fp);

    return 0;
}

```

Reading a Character Using fgetc()

```

#include <stdio.h>

int main()
{
    FILE *fp;

    char ch;

    fp = fopen("data.txt", "r");

    if (fp == NULL)

        return 1;

    while ((ch = fgetc(fp)) != EOF)
    {
        printf("%c", ch);
    }

    fclose(fp);

    return 0;
}

```

Important Points

- Writing a file means storing information generated or entered by the program into a permanent storage file.
- Reading a file means retrieving previously stored information from the file into the program.
- The fputc() function is used to write one character at a time into a file.
- The fgetc() function is used to read one character at a time from a file.
- The fputs() function is used to write a complete string into a file.
- The fgets() function is used to read a string or a line from a file.
- The EOF value is used to indicate that the end of a file has been reached while reading.
- A file opened using read mode should normally be used only for retrieving existing information.

- A file opened using write or append mode can be used to store new information.
- Character and string functions are useful when the program needs to process textual information stored in a file.

Processing a Data File

File processing refers to performing operations on the information stored in a file. A program can read records, search for particular information, calculate values, count data, update records, or copy information from one file to another.

Example – Count Characters

```
#include <stdio.h>

int main()
{
    FILE *fp;
    char ch;
    int count = 0;
    fp = fopen("data.txt", "r");
    if (fp == NULL)
    {
        printf("File cannot be opened.");
        return 1;
    }
    while ((ch = fgetc(fp)) != EOF)
    {
        count++;
    }
    printf("Number of characters = %d", count);
    fclose(fp);
    return 0;
}
```

Important Points

- File processing involves performing useful operations on information stored inside a file.
- A program can process a file by reading its contents one character, line, or record at a time.
- Searching a file allows the program to locate a particular record or value.

- Counting operations can be used to determine the number of characters, words, lines, or records in a file.
- A program can calculate values such as total marks or average marks using information stored in a file.
- File processing can be used to copy the contents of one file into another file.
- Files containing multiple records can be processed sequentially from the beginning to the end.
- A program can compare information stored in different files during file processing.
- File processing is commonly used in applications that maintain permanent records.
- Proper file opening, error checking, processing, and closing are important steps in a file-processing program.

File Handling Functions

C provides a collection of library functions for performing different operations on files. These functions are declared in the `stdio.h` header file.

Important File Functions

Function Explanation

`fopen()` Opens a file for a specified operation.

`fclose()` Closes an opened file.

`fgetc()` Reads one character from a file.

`fputc()` Writes one character into a file.

`fgets()` Reads a string or line from a file.

`fputs()` Writes a string into a file.

`fprintf()` Writes formatted data into a file.

`fscanf()` Reads formatted data from a file.

`fread()` Reads binary data from a file.

`fwrite()` Writes binary data into a file.

`fseek()` Moves the file pointer to a specified position.

`ftell()` Returns the current position of the file pointer.

`rewind()` Moves the file pointer back to the beginning.

`feof()` Checks whether the end of the file has been reached.

Function Explanation

`ferror()` Checks whether an error has occurred during file processing.

Important Points

- The `fopen()` function is used to establish a connection between the program and a file.
- The `fclose()` function terminates the connection between the program and the opened file.
- The `fgetc()` function reads one character at a time from a file.
- The `fputc()` function writes one character at a time into a file.
- The `fgets()` and `fputs()` functions are mainly used for reading and writing strings.
- The `fprintf()` and `fscanf()` functions are used when formatted data needs to be written or read.
- The `fread()` and `fwrite()` functions are mainly used for handling binary data.
- The `fseek()` function allows the program to move the file pointer to a particular location.
- The `ftell()` and `rewind()` functions help the program determine and change the current file position.
- Functions such as `feof()` and `ferror()` help the program identify end-of-file conditions and file-related errors.

Formatted Input and Output Functions

Formatted file input and output functions allow data of different types to be stored and retrieved according to a specified format. The two main functions are `fprintf()` and `fscanf()`.

`fprintf()`

The `fprintf()` function is used to write formatted information into a file.

```
fprintf(fp, "%d %s %.2f", roll, name, mark);
```

`fscanf()`

The `fscanf()` function is used to read formatted information from a file.

```
fscanf(fp, "%d %s %f", &roll, name, &mark);
```

Example

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *fp;
```

```
    int roll = 101;
```

```
    char name[] = "Anu";
```

```

float mark = 89.5;

fp = fopen("student.txt", "w");

if (fp == NULL)

    return 1;

fprintf(fp, "%d %s %.2f", roll, name, mark);

fclose(fp);

return 0;

}

```

The file will contain:

101 Anu 89.50

Important Points

- Formatted file input and output allow different types of data to be handled according to specified format strings.
- The `fprintf()` function writes formatted information into a file.
- The `fscanf()` function reads formatted information from a file.
- The first argument of `fprintf()` and `fscanf()` is the file pointer associated with the file.
- Format specifiers such as `%d`, `%f`, `%c`, and `%s` specify the type of data being processed.
- The `fprintf()` function works similarly to `printf()`, but its output is directed to a file.
- The `fscanf()` function works similarly to `scanf()`, but its input is obtained from a file.
- Formatted file functions are useful for storing structured records such as student or employee details.
- The address operator `&` is generally required for numeric variables when they are passed to `fscanf()`.
- Formatted file processing makes it easier to store and retrieve records containing multiple data types.

File Processing Programs

File processing programs are programs designed to perform specific operations on files. These programs are commonly used to maintain permanent records in real-world applications.

Example – Student Record

```

#include <stdio.h>

int main()

{

```

```

FILE *fp;

int roll;

char name[30];

float mark;

fp = fopen("student.txt", "w");

if (fp == NULL)
{
    printf("File cannot be opened.");
    return 1;
}

printf("Enter Roll Number: ");

scanf("%d", &roll);

printf("Enter Name: ");

scanf("%s", name);

printf("Enter Mark: ");

scanf("%f", &mark);

fprintf(fp, "%d %s %.2f\n", roll, name, mark);

fclose(fp);

printf("Student record stored successfully.");

return 0;
}

```

Important Points

- A file processing program is designed to perform one or more operations on stored file data.
- Student records can be stored in files so that they remain available after the program terminates.
- Employee information can be maintained permanently using file-processing programs.
- A program can read existing records and display them to the user.
- A search operation can be implemented to find a particular record from a file.
- New records can be added to an existing file using append mode.
- File-processing programs can calculate totals, averages, maximum values, and minimum values from stored records.

- A program can copy the contents of one file into another file using character or block operations.
- File-processing techniques form the basic foundation for many record-management applications.
- Error checking should be performed whenever a file is opened, read, written, or closed.

Concept of Binary Files

A binary file stores data in a binary representation that is suitable for direct processing by a computer. Unlike a text file, the contents of a binary file are generally not intended to be directly read by humans.

Example

```
#include <stdio.h>

struct Student
{
    int roll;
    char name[30];
    float mark;
};

int main()
{
    FILE *fp;
    struct Student s = {101, "Anu", 89.5};
    fp = fopen("student.dat", "wb");
    if (fp == NULL)
        return 1;
    fwrite(&s, sizeof(s), 1, fp);
    fclose(fp);
    return 0;
}
```

Reading the Binary File

```
#include <stdio.h>

struct Student
{

```

```

int roll;

char name[30];

float mark;
};

int main()
{
    FILE *fp;

    struct Student s;

    fp = fopen("student.dat", "rb");

    if (fp == NULL)

        return 1;

    fread(&s, sizeof(s), 1, fp);

    printf("Roll Number = %d\n", s.roll);

    printf("Name = %s\n", s.name);

    printf("Mark = %.2f\n", s.mark);

    fclose(fp);

    return 0;
}

```

Important Points

- A binary file stores information in a binary representation rather than as human-readable characters.
- Binary files are particularly useful for storing structures and other types of structured data.
- The fwrite() function is used to write binary data into a file.
- The fread() function is used to retrieve binary data from a file.
- The wb mode is used when a binary file needs to be opened for writing.
- The rb mode is used when a binary file needs to be opened for reading.
- Binary files can store a structure as a block of bytes using its size obtained through sizeof().
- Binary files are generally not convenient for direct viewing or editing using ordinary text editors.
- Binary file processing can be efficient when large numbers of structured records need to be stored and retrieved.

- The same structure definition and compatible data representation should be used when reading back records stored in a binary file.

Enumeration

Enumeration is a user-defined data type in C that allows a programmer to assign meaningful names to a set of integer constants. It is declared using the keyword `enum`.

Syntax

```
enum Day
{
    SUNDAY,
    MONDAY,
    TUESDAY,
    WEDNESDAY
};
```

By default:

```
SUNDAY = 0
MONDAY = 1
TUESDAY = 2
WEDNESDAY = 3
```

Example

```
#include <stdio.h>

enum Result
{
    FAIL = 0,
    PASS = 1
};

int main()
{
    enum Result result;
    result = PASS;
    if (result == PASS)
        printf("Student Passed");
    return 0;
```

}

Important Points

- Enumeration is a user-defined data type that represents a collection of named integer constants.
- The keyword `enum` is used to declare an enumeration type in C.
- Enumeration makes programs easier to understand because meaningful names can be used instead of numeric values.
- By default, the first enumeration constant receives the value zero.
- Each following enumeration constant normally receives a value one greater than the previous constant.
- A programmer can explicitly assign specific integer values to enumeration constants.
- Enumeration is useful for representing a fixed set of related choices such as days, months, states, or results.
- An enumeration variable can be declared using the name of the enumeration type.
- Using meaningful enumeration names can improve the readability and maintainability of a program.
- Enumeration is especially useful when a program needs to represent a variable that can logically have one value from a predefined set.

Bitwise Operators

Bitwise operators perform operations directly on the individual bits of integer values. These operators are important in low-level programming, embedded systems, operating systems, and applications involving flags.

Main Operators

Operator Meaning

& Bitwise AND

、 、

^ Bitwise XOR

~ Bitwise NOT

<< Left Shift

>> Right Shift

Example

Consider:

5 = 0101

3 = 0011

AND:

0101

0011

0001

Therefore:

5 & 3

produces:

1

Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 5;
```

```
    int b = 3;
```

```
    printf("AND = %d\n", a & b);
```

```
    printf("OR = %d\n", a | b);
```

```
    printf("XOR = %d\n", a ^ b);
```

```
    printf("Left Shift = %d\n", a << 1);
```

```
    printf("Right Shift = %d\n", a >> 1);
```

```
    return 0;
```

```
}
```

Important Points

- Bitwise operators perform operations directly on the binary representation of integer values.
- The bitwise AND operator & produces one only when the corresponding bits of both operands are one.
- The bitwise OR operator | produces one when at least one of the corresponding bits is one.
- The bitwise XOR operator ^ produces one when the corresponding bits of the operands are different.
- The bitwise NOT operator ~ reverses each bit of its operand.

- The left-shift operator << moves the bits toward the left by a specified number of positions.
- The right-shift operator >> moves the bits toward the right by a specified number of positions.
- Bitwise operators are useful for setting, clearing, checking, and modifying individual bits.
- Bitwise operations are commonly used in systems programming, embedded programming, device control, and flag management.
- Bitwise operators should not be confused with logical operators such as && and ||, because they operate at the bit level rather than the logical-condition level.

Command Line Parameters

Command line parameters allow information to be supplied to a program when the program is started. They are received through the parameters argc and argv of the main() function.

Syntax

```
int main(int argc, char *argv[])
```

Here, argc represents the number of command line arguments and argv is an array of strings containing those arguments.

Example

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i;

    printf("Number of arguments = %d\n", argc);

    for (i = 0; i < argc; i++)
    {
        printf("Argument %d = %s\n", i, argv[i]);
    }

    return 0;
}
```

If executed as:

```
program.exe MCA Student
```

the arguments are approximately:

```
argc = 3
```

```
argv[0] = program.exe
```

```
argv[1] = MCA
```

```
argv[2] = Student
```

Example – Addition

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    int a, b;
```

```
    if (argc != 3)
```

```
    {
```

```
        printf("Please provide two numbers.");
```

```
        return 1;
```

```
    }
```

```
    a = atoi(argv[1]);
```

```
    b = atoi(argv[2]);
```

```
    printf("Sum = %d", a + b);
```

```
    return 0;
```

```
}
```

Important Points

- Command line parameters allow values to be supplied to a program during program execution.
- The argc parameter stores the number of command line arguments received by the program.
- The argv parameter stores the command line arguments as an array of character strings.
- The first element argv[0] normally contains the name or path of the program being executed.
- The actual user-supplied arguments normally begin from argv[1].
- Command line arguments are initially received as strings even when they represent numeric values.
- Functions such as atoi() can be used to convert a numeric string into an integer.
- The value of argc should be checked before accessing specific elements of argv.
- Command line parameters are useful for programs that need input without asking the user interactively.
- Command line arguments are commonly used in utilities, automation scripts, testing programs, and system-level applications.

Macros

A macro is a name or piece of code defined using the C preprocessor. Macros are processed before the actual compilation of the program begins.

The `#define` directive is commonly used to create macros.

Example

```
#define PI 3.14159
```

Whenever `PI` occurs in the program, the preprocessor replaces it with `3.14159`.

Program

```
#include <stdio.h>
```

```
#define PI 3.14159
```

```
int main()
```

```
{
```

```
    float r = 5;
```

```
    printf("Area = %.2f", PI * r * r);
```

```
    return 0;
```

```
}
```

Important Points

- A macro is a symbolic name or code fragment that is processed by the C preprocessor before compilation.
- The `#define` directive is used to define a macro in a C program.
- Macros are replaced by the preprocessor before the compiler processes the program.
- Macros can be used to define constant values that are required at several places in a program.
- Macros can also be used to represent expressions or small pieces of reusable code.
- A macro definition normally does not require a semicolon at the end.
- Macros can improve readability by replacing difficult or repeated values with meaningful names.
- Macros are different from variables because macro replacement occurs during preprocessing rather than during program execution.
- Function-like macros can accept parameters and generate expressions based on those parameters.
- Careful use of parentheses is important in function-like macros to avoid unexpected results due to operator precedence.

Function Macros

A function macro is a macro that accepts one or more arguments and produces a replacement expression. It is also called a function-like macro because its syntax resembles a function call.

Example

```
#define SQUARE(x) ((x) * (x))
```

Program:

```
#include <stdio.h>
```

```
#define SQUARE(x) ((x) * (x))
```

```
int main()
```

```
{
```

```
    int n = 5;
```

```
    printf("Square = %d", SQUARE(n));
```

```
    return 0;
```

```
}
```

Output:

Square = 25

Important Points

1. A function-like macro is a macro that accepts one or more arguments.
2. The parameter of a function macro is replaced by the corresponding argument before compilation.
3. A function macro is invoked using syntax that looks similar to a normal function call.
4. Function macros can be used to perform simple calculations or repeated expressions.
5. The SQUARE(x) macro can calculate the square of a value without defining a separate function.
6. Parentheses should be used around macro parameters to avoid incorrect results when expressions are supplied as arguments.
7. The complete macro expression should also normally be enclosed in parentheses.
8. Unlike a normal function, a macro does not perform type checking on its arguments in the same way.
9. A macro argument can potentially be evaluated more than once, so expressions with side effects should be used carefully.

10. Function macros are useful for small, simple operations where preprocessor substitution is appropriate.

Conditional Macros

Conditional macros are used to control which portions of a program are included during compilation. This technique is called conditional compilation.

The important directives include:

`#if`

`#ifdef`

`#ifndef`

`#else`

`#elif`

`#endif`

Example – `#ifdef`

```
#include <stdio.h>
```

```
#define DEBUG
```

```
int main()
```

```
{
```

```
#ifdef DEBUG
```

```
    printf("Debug mode is enabled.\n");
```

```
#endif
```

```
    printf("Program executed.");
```

```
    return 0;
```

```
}
```

Example – `#ifndef`

```
#ifndef MAX
```

```
#define MAX 100
```

```
#endif
```

This means that MAX is defined as 100 only if it has not already been defined.

Example – `#if`

```
#include <stdio.h>
```

```
#define VERSION 2
```

```
int main()
```

```
{  
#if VERSION == 1  
    printf("Version 1");  
#else  
    printf("Version 2");  
#endif  
    return 0;  
}
```

Important Points

- Conditional compilation allows different parts of a C program to be included or excluded before compilation.
- The #if directive includes code when a specified preprocessor condition is true.
- The #ifdef directive checks whether a particular macro has already been defined.
- The #ifndef directive checks whether a particular macro has not been defined.
- The #else directive provides an alternative section when the preceding condition is false.
- The #elif directive allows another condition to be checked when previous conditions are false.
- The #endif directive marks the end of a conditional compilation block.
- Conditional compilation is useful for creating different versions of a program for different environments.
- Debugging statements can be included using a macro such as DEBUG and removed when debugging is disabled.
- Conditional macros are also widely used in header files to prevent the same header file from being included multiple times.